Available online at www.sciencedirect.com

ScienceDirect

Procedia Computer Science 233 (2024) 138–146

Procedia Computer Science

www.elsevier.com/locate/procedia

5th International Conference on Innovative Data Communication Technologies and Application (ICIDCA 2024)

A Multithreaded-BFS Algorithm for Optimizing the Graph Computation in Multicore Processing System

Kapil Juneja[a], Dhiraj Khurana[b]

[a]*Department of Computer Science Engineering, Bennett University, Greater Noida, Uttar Pradesh*
[a]*kapil.juneja81@gmail.com*
[b]*Department of Computer Science Engineering, Maharshi Dayanand University, Rohtak, Haryana*
[b]*dhirajkhurana34@gmail.com*

Abstract

Graphs are the most complex data structure that involves heavy mathematical computations. Various real-time applications use the graph at storage, processing, object characterization, and behavior specification stages. The connectivity analysis, coverage building, and routing are the critical applications of graph evaluation. Various associated applications need the exploration of a graph with a practical and real-time outcome. The processing of larger graphs is a big challenge for the researcher. In this paper, a hybrid graph-based computation optimization method is designed to optimize the performance of graph processing in multi-core processors. The BFS algorithm is applied over the multiple processors to optimize the functional response. The multithreaded-BFS is implemented to reduce the time of graph processing and computation. The proposed algorithm is implemented in the OpenMP environment. The experimentation is done on two-core and four-core processors. The experiment is conducted on a 100 to 1000-node network. The traversing and graph coloring algorithms are implemented for the analysis. The comparative analysis is conducted on single-core and multi-core processors with DFS and BFS-based algorithms. The analysis results identified that the proposed algorithm reduced the processing time effectively.

© 2024 The Authors. Published by Elsevier B.V.
This is an open access article under the CC BY-NC-ND license (https://creativecommons.org/licenses/by-nc-nd/4.0)
Peer-review under responsibility of the scientific committee of the 5th International Conference on Innovative Data Communication Technologies and Application

Keywords: Breadth-First Search; Depth First Search; Graph Processing; Parallel Processing; Multi-Core Processor

1. Introduction

The processing speed and efficiency [1, 2] are the key challenges of any software, hardware, application, environment, or algorithm. Collaborative communication and behavior between these performance-driven components are required to optimize communication. At the higher level, an application, framework, or component

1877-0509 © 2024 The Authors. Published by Elsevier B.V.
This is an open access article under the CC BY-NC-ND license (https://creativecommons.org/licenses/by-nc-nd/4.0)
Peer-review under responsibility of the scientific committee of the 5th International Conference on Innovative Data Communication Technologies and Application
10.1016/j.procs.2024.03.203

is available to the user. At the lower level, the hardware components and algorithmic approaches affect the performance and efficiency of an element, tool, application, and environment. At the hardware level, the processing capability of the CPU or microcontroller affects the performance of the complete system. The processing unit, cache memory, DMA, and pipelining are the common elements of a processor that affect the performance of the complete system or application. The principal element is the number of cores. A single core system defines an entire individual processing unit with a separate Arithmetic and Logical Unit (ALU), control unit (CU), memory unit (MU), and cache. In a multi-core system, multiple processing units can handle parallel processing at the hardware level [3, 4, 5]. Figure 1 shows the standard architecture of a multi-core processor.

Multi-core systems can be used to solve the complex problem for which real parallel processing is required. The significance of the micro-core system is mainly identified by the problems of having a larger data size. The graph and network-based processing system also use multi-core functionalities to improve performance and to provide the result in real-time. A complex problem that can be divided into smaller tasks or functional segments can be processed by each core separately in these systems. Such systems optimize the operational capability by distributing the load among different cores. Resource utilization is the key capability of multi-core systems to improve the behavior or processing of an application environment, or system [6, 7, 8].

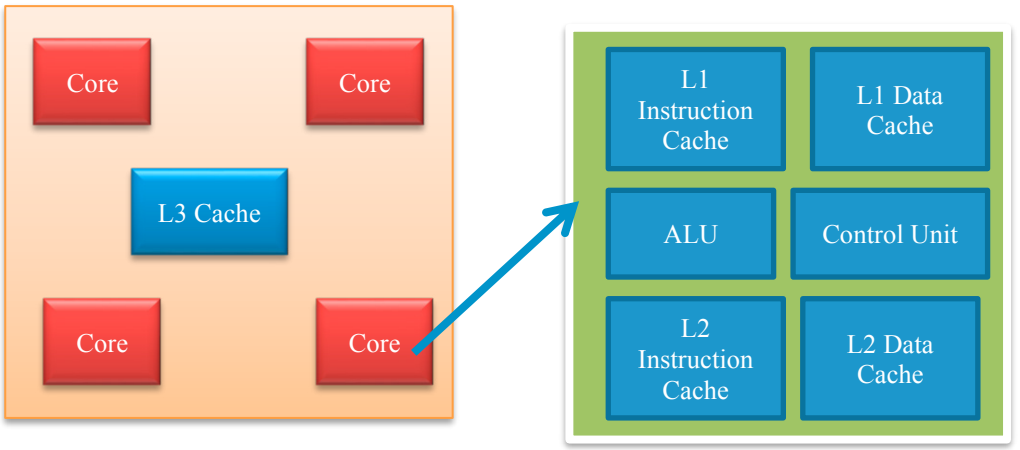


Figure 1: Multi-core Processor

In small or middle-level applications, the functional process is divided into smaller threads to run in parallel. These threads are still handled by a single processor in a time-slicing mechanism. However, as the complexity of the problems increases, these single processor threads are not enough to provide a real-time parallel solution. A core-specific thread handling can be adapted to provide real parallelism in the real environment. In IoT-based real-time environment [9], GIS systems [10], graph-based processing [11], multi-model biometric [12] systems are some of such real-time applications that require parallelism in threads to derive the outcome in the significant time.

Graphs are one global, vast, and multi-aspect complex problem that exists at different levels based on their usage and involvement. The graphs can be used as a storage system, network, graphics, image-based system, and resource allocation system. There are varieties of applications and environments that cannot be imagined without the involvement of graphs [13, 14] Website access, internet usage, task allocation, and scheduling are common applications of graphs. As the number of components of nodes in a graph increase, the number of connectivity goes exponentially. In such a case, graph processing requires a lot of resources and effort to generate an outcome in real-time. Most of the graph processing problems come under NP-Complete and NP-Hard. This paper investigates a multi-threaded BFS method to optimize graph processing in multi-core processors [15, 16, 17].

1.1. Research Contributions

A graph is a more critical and complex structure that is been used in social networks, communication systems, and all kinds of resource-level distribution networks. Many of the graph operations come under NP-complete and NP-Hard problems. The multicore processor uses the complete graph in an application that does not utilize the parallel processing provided by multi-core processors. The main contributions of the proposed work are given below:

- The paper presented a multi-core BFS (Best First Search) algorithm to divide the graph into smaller segments and divided the functional task of the main task into smaller subtasks. Each subtask is assigned to a separate core for multi-core parallel processing.
- The dynamic division of sub-problems and combining them as the operation is applied over it. The results are collected from individual cores, combined, and presented as the outcome.
- The proposed algorithm has experimented with two major graph operations called graph traversing and graph coloring problems.
- The algorithm is analyzed against the scalability factor with 100 to 1000 node graphs.
- The comparative analysis is done against two-core and 4-core processors with existing depth-first search and breath-first search algorithms.

This section defines the primary application, problems, and significance of multi-core processors. The exploration of internal components also defines the processor-level architecture. Graph processing is a complex problem and application area for a multi-core system. In section 2, the work investigated by the researchers on multi-core systems and graph problems is defined. In section 3, the proposed multi-threaded BFS algorithm for multi-core systems. In section 4, the implementation results on complex graphs are analyzed respectively for the multi-core and single-core processors. The comparative analysis results are provided to validate the work against traversing and graph coloring algorithms. In Section 5, the conclusion and future scope of this work.

2. Related Work

The paper has provided an optimized algorithm to use the capabilities of multicore processors to improve the performance of graph algorithms. The complex graph problems are non-polynomial by nature. Many algorithms, mathematical models, and optimization methods were investigated by the researchers to optimize graph processing for real-time applications.

Graph defines the storage behaviour, structural aspects, and connectivity for various science and engineering applications. Computer networks, social connectivity, and multimedia applications use the graph as a key phenomenon for establishing the relationship between resources and optimizing storage behaviour. The core components of the graph are vertices and edges. The vertices are the data elements or nodes, and edges represent the connectivity or relationships between these data elements. The nodes and connectivity are defined with different features respective to the application and environment. Different kinds of graphs are formed with different processor architectures to optimize the functioning and behaviour of the particular application or environment. Sparse graphs, connected graphs, and complete graphs are the common graph types for different types. The traversing, shortest route, and graph colouring are the key applications used in different applications to optimize the performance of graphs. [18, 19] provided a study and work on graph processing and traversing methods. A sparse graph is one such common graph type used in many networks including sparse networks. The sparse network with lesser connectivity is shown in Figure 2. The graphs are provided here with different densities N=5, 10, and 20.

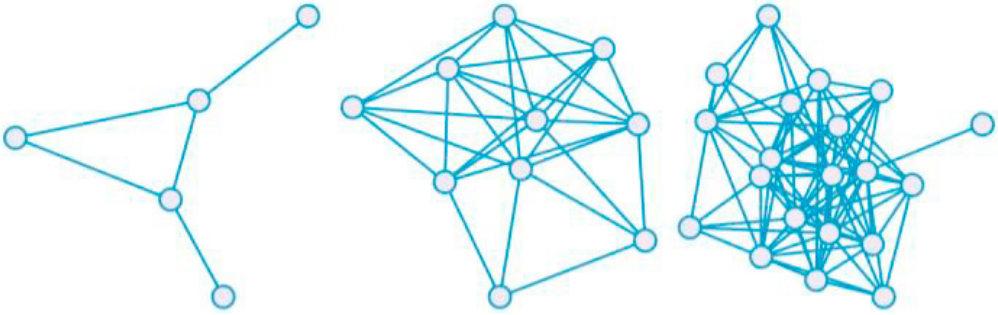


Figure 2: Sparse Graphs with different Density Ratio

It shows that as the graph density and connectivity increase, the algorithmic complexity and challenges also increase. In such cases, the algorithmic models are required to reduce the processing time. The multiprocessing and threading systems were used to optimize the performance of large graph processing. The programming adaptive models were also used to optimize the behavior and functionality of graphs in different application environments. A three-stage model is defined to process the graph in a real environment by Agarwal et al. [19]. Figure 3 the functional process of this three-phase graph processing. The function stages of the graph processing method are the Pruning, Process, and Post-processing stages. In the first stage, filtration is applied over the graph by removing the disconnected nodes and components. The irrelevant components are mapped and identified to optimize the performance of the model. The effectiveness of the pruning stage defines the reliability of the system. Once the graph with validated information is available, the actual algorithm is processed over it. The algorithm uses the constraints, features, and behavior of the application area and environment [20, 21]. Finally, post-processing is defined to combine the results of the sub-graph and present them for an original large graph. This model is shown in Figure 2.

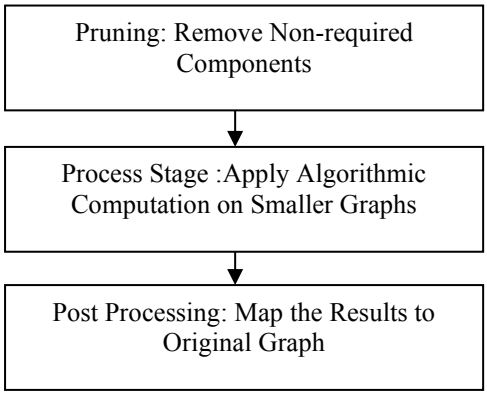


Figure 3: A standard three-stage graph processing framework [19]

Various graph optimization algorithms were used to improve the reliability and performance of graph traversing. The BFS integration within parallel programming [22] methods were done to improve the reliability of the application environment. The method was applied to a scalable graph with a multi-core processor environment. The method applied irregular graphs to optimize the performance of parallel computing. The analysis results identified the proposed algorithm reduced the computational time in a processor-specific experiment.

3. Research Methodology

The paper has presented a multithreaded-BFS-based method for enabling parallel computing in a real application environment effectively. As discussed in the previous section, the objective of the system is to divide a complex task into smaller subtasks and execute them on different cores separately as individual threads. The proposed model is implemented on a quad-core processor and applied to the larger graphs. The experimental study is conducted on graph traversing and coloring problems. The experimental study related to the work is provided in section 4. Table 1 shows the functional stages of the proposed Multithreaded-BFS algorithm for graph traversing in multi-core processors.

The proposed algorithm is applied on a graph G(V, E) is defined with two sets of vertices (V) and Edges (E). The graph is illustrated with an initial or start node (S). S is the component of set V. The graph traversing is started from node S and will visit all graph nodes precisely one time. The graph is defined as a connected structure or matrix in which one represents the edge existence between node pairs, and 0 means no connectivity exists between the nodes. P is another list that contains information about the parent node. The parent is the previously connected node of the current observing node. To enable the traversing, two shared lists of queues are maintained and updated by each core. These queues are the CQ to store the vertices of the current level being processed. NQ is the queue that stores the vertices of the next level. In the first stage of this algorithm, the complete graph is divided into M number of segments with K nodes. M depends on the number of cores. The centralized controller performs and manages the virtual allocation, and information is maintained in the centralized cache. Nc is the allocated node to a specific core.

Table 1 shows the operational process taken by each core for traversing the allocated graph nodes. Each core node is processed and maintained by the parent nodes in the Parent List, and visiting status is maintained. The process starts from node S, identifies the nodes at the current level, and is kept in the CQ list. A separate thread performs a deep check for each current node to identify the nodes in the next levels. This next-level list is maintained in NQ. After processing the next level list, the elements of CQ are processed in parallel by a separate thread. Each element from CQ is extracted, and the separate thread identifies the neighbors of current nodes. If the extracted vertex is not visited, then set its status to visited and include it in the visited queue list. This process is continued for each core in parallel, and the parallel threads process the nodes of the next and current levels. The process will be repeated till all nodes of each core are not processed.

Table 1 : Multithreaded-BFS-based Graph Processing Algorithm

```
Algorithm(G,Nc,S,CQ,NQ, Status,P)
/*G is the graph representation with vertices set V and edge set E.
Nc is the number of nodes allocated to a specific core
S is the root vertex from where the traversing will begin.
P is the array to store the parent of nodes.
CQ is the queue to store vertex in the current level.
NQ holds the vertices for the next level. Status is the status list that represents the visiting status of nodes*/
Begin
1.      For i=1 to Nc
        [Process All nodes]
2.      P(i)=Inf
        Status(i)=0
        [Initially, no node is visited so the status is set at 0 and no parent is identified for any node]
        End If
3.      P(S)=0
        [No parent will be defined for root node]
4.      Insert(CQ,S)
```

```
    [Add the root element to CQ as the first processing element of the graph]
5.  While (CQ<>Null)
    [Repeat process till all Queue elements are not processed]
    Begin
6.  Set NQ=Null
    [Empty the NQ for Next level processing]
7.  While (CQ<>Null)
    [Process all Elements of CQ under parallel computing]
    Begin
8.  u=GetElement(CQ)
    [Remove the element from CQ and present it as Current Element]
9.  neigh=GetNeighbors(u)
    [Get the adjacent to current node u ]
10. for i=1 to neigh. Length
    [Process all adjacent to node u]
    Begin
11. v=neigh(i)
    status=Status(v)
    [Get the visiting status of neighbour nodes]
12. if(status=0)
    [Process the neighbour node that is not already visited]
    Begin
13. prev=Status(v)
    Status(v)=1
    [Get the visiting node status in prev and visit the node]
14. if (prev=0)
    [If the node is not visited]
    Begin
15. Set Parent(v)=u
    [Set the current node as parent to neighbour node]
16. Insert(NQ,v)
    [Insert the neighbour node in the queue]
    End If
    End If
    End For
    End While
17. Interchange(CQ,NQ)
    [Set the Next level queue as the Current Level Queue]
    End While
    End
```

4. Results and Analysis

The paper has presented a multithreaded-BFS algorithm for optimizing the functional processing of complex applications in a multi-core environment. In this section, the experimentation of the proposed algorithm is done on complex and more extensive graphs. The graph traversing and coloring applications are considered in the experiments. The experiment is conducted on a single-core, dual-core, and quad-core processor. The standard DFS and BFS algorithms have been experimented with as state-of-the-art methods in these environments. The proposed multithreaded-BFS algorithm is implemented in a quad-core environment. These algorithms and environments are simulated in the OpenMP platform and configured with the Intel Xeon cluster. The processor is defined with a 2.6Hz clock speed, 16GB DDR RAM, and 20M cache memory. This OpenMP environment is integrated with C++ and g++ libraries. The experiments are conducted on larger graphs with 50 to 5000 vertices. The performance analysis results of graph traversing are provided in Table 2. The analysis results are taken in processing time (in seconds). The table clearly shows that the proposed method effectively reduced the processing time compared to single-core, dual, and conventional DFS and BFS algorithms. Figure 4 shows the experiment results graphically.

Table 2: Performance Analysis (Graph Traversing)

No. of Nodes	Single Core+ DFS	Dual Core +DFS	Quadcore+DFS	Single Core + BFS	Dual Core + BFS	Proposed
50	4.23	4.020809	3.817366	3.302591	2.383281	2.00203
100	10.18	9.14562	7.234787	7.060346	5.572346	5.202305
200	17.63	16.3358	14.97939	10.36506	8.137368	7.055618
500	39.46	32.74757	31.76574	27.71156	26.97304	24.47531
1000	76.15	67.97125	60.55032	49.37612	48.20041	42.7768
5000	238.14	216.7585	196.0277	193.0463	183.4188	156.7679

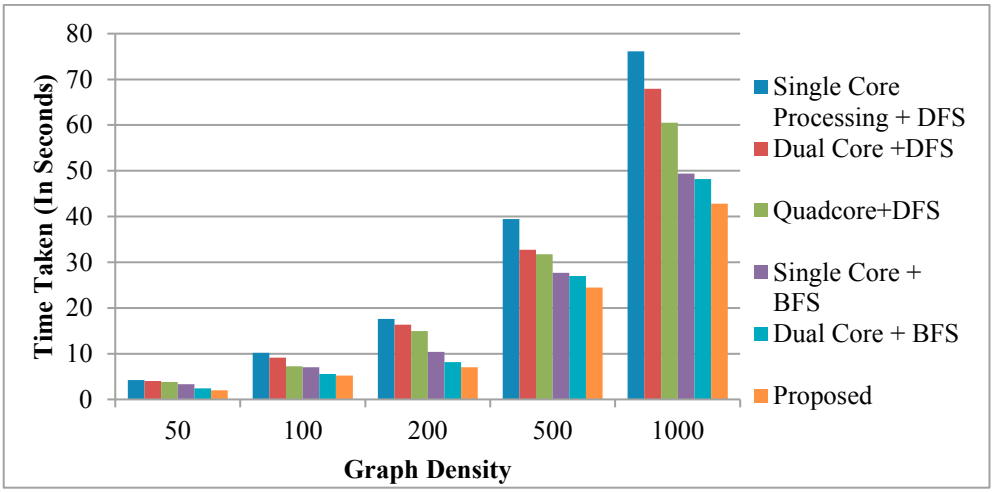


Figure 4: Task Completion Time Analysis (Graph Traversing)

Figure 4 shows the comparative evaluation of the proposed multithreaded-BFS against conventional methods. The bar graph clearly shows that the proposed algorithm effectively reduces the processing time against existing methods. The experiments are also conducted on different graphs for graph coloring problems. The performance analysis results for graph coloring are provided in Table 3 and Figure 5 against the existing methods. The analysis results show that the proposed model reduced the processing time extensively, and results were obtained in less

time. The bar graph of Figure 5 shows that the proposed multithreaded-BFS algorithm has improved the performance of the system, and the results are achieved effectively in less time than the existing methods.

Table 3: Performance Analysis (Graph Coloring)

No. of Nodes	Single Core + DFS	Dual Core +DFS	Quadcore+DFS	Single Core + BFS	Dual Core + BFS	Proposed
50	5.901345	5.238671	5.544931	4.320307	3.204671	2.689588
100	14.48084	12.82866	10.54908	10.37757	8.244451	6.927821
200	24.58585	22.06285	20.68402	13.24427	11.88813	10.12451
500	50.83749	43.22396	41.46853	41.55341	38.01615	34.84457
1000	113.7931	97.41092	83.00066	62.45926	62.41787	54.36578
5000	300.4726	288.3356	263.6758	247.2013	228.7558	185.7286

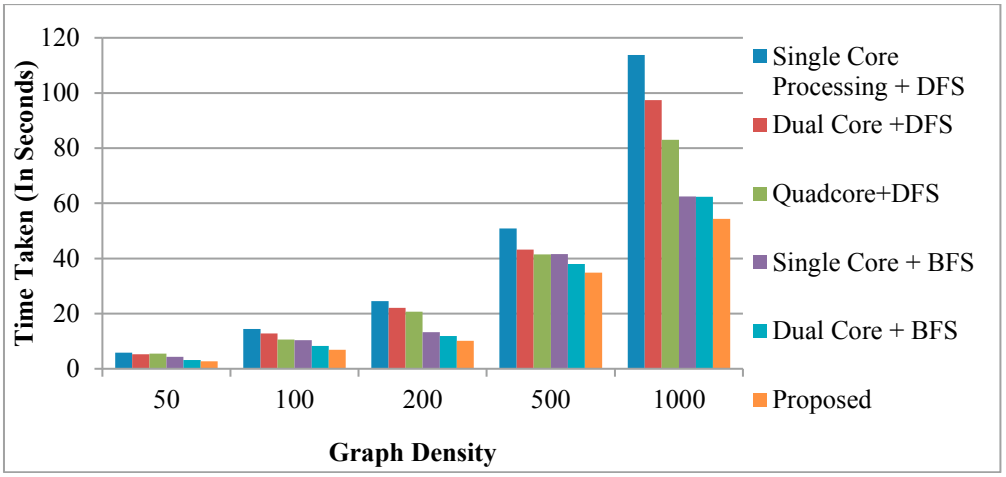


Figure 5: Task Completion Time Analysis (Graph Coloring)

Figure 5 provides the comparative results of graph coloring with 50 to 1000 nodes. For the larger graph with 1000 nodes, the single-core processor with DFS takes over 300 seconds, and the proposed multi-threaded BFS algorithm takes upto 185 seconds. It shows that the proposed algorithm in a quad-core environment has achieved task-based parallel processing and effectively reduced processing speed.

4. Conclusion

Parallel processing is accomplished at the processor, architecture, and algorithm levels to handle complex problems. In this paper, the graph is considered a complex application that takes much time in a single processor or single-core system. The paper has presented a multithreaded-BFS algorithm for a multi-core processor. The larger graphs are processed partially on each core as a separate thread in this algorithm. The experiments are conducted on graph traversing and coloring problems. The analysis results identified that the proposed algorithm reduced the processing time against existing single-core and multi-core-based conventional algorithms.

References

[1] M. W. Hassan, P. M. Athanas and Y. Y. Hanafy, "Domain-Specific Modeling and Optimization for Graph Processing on FPGAs," in *International Symposium on Applied Reconfigurable Computing*, 2021.

[2] E. H. Wu and M. K. Ng, "A Graph-Based Optimization Algorithm for Website Topology Using Interesting Association Rules," in *Pacific-Asia Conference on Knowledge Discovery and Data Mining*, 2003.

[3] P. Lutsyk, J. Oberhauser and W. J. Paul, "Multi Core Processors," *A Pipelined Multi-Core Machine with Operating System Support,* pp. 489-504, 2020.

[4] M. Balakrishnan, "Power Consumption in Multi-core Processors," in *International Conference on Contemporary Computing*, 2012.

[5] R. Suraj, P. Chitra and S. Madumidha, "Evaluation of Worst-Case Execution Time of Tasks on Multi-core Processor," *Artificial Intelligence and Evolutionary Algorithms in Engineering Systems,* pp. 441-448, 2014.

[6] A. Vajda, "Multi-core and Many-core Processor Architectures," in *Programming Many-Core Chips*, 2011.

[7] C. Ababei and M. G. Moghaddam, "A survey of prediction and classification techniques in multi-core processor systems," *IEEE Transactions on Parallel and Distributed Systems,* vol. 30, no. 5, pp. 1184-1200, 2018.

[8] M. Schoeberl, L. Pezzarossa and J. Spars, "A multi-core processor for time-critical applications," *IEEE Design & Test,* vol. 35, no. 2, pp. 38-47, 2018.

[9] Y. N. Malek, A. Kharbouch, H. E. Khoukhi, M. Bakhouya, V. D. Florio, D. E. Ouadghiri, S. Latré and C. Blondia, "On the use of IoT and big data technologies for real-time monitoring and data processing," *Procedia computer science,* vol. 113, pp. 429-434, 2017.

[10] E. P. Blasch, P. B. Deignan, S. L. Dockstader, M. Pellechia, K. Palaniappan and G. Seetharaman, "Contemporary Concerns in Geographical/Geospatial Information Systems (GIS) Processing," in *IEEE National Aerospace and Electronics Conference (NAECON)*, 2011.

[11] A. Koller, S. Oepen and W. Sun, "Graph-based meaning representations: Design and processing," in *57th Annual Meeting of the Association for Computational Linguistics: Tutorial Abstracts*, 2019.

[12] S. S. Omran and M. A. Salih, "Design and implementation of multi-model biometric identification system," *International Journal of Computer Applications,* vol. 99, no. 15, pp. 14-21, 2014.

[13] V. Balaji and B. Lucia, "When is graph reordering an optimization? studying the effect of lightweight graph reordering across applications and input graphs," in *IEEE International Symposium on Workload Characterization (IISWC)*, 2018.

[14] X. Zhou, P. Chang and G. Chen, "An Efficient Graph Processing System," in *Asia-Pacific Web Conference*, 2014.

[15] H. Cai, V. W. Zheng and K. C.-C. Chang, "A comprehensive survey of graph embedding: Problems, techniques, and applications," *IEEE Transactions on Knowledge and Data Engineering,* vol. 30, no. 9, pp. 1616-1637, 2018.

[16] L. Yang, C. Zhong, Q. Yang, W. Zou and A. Fathalla, "Task offloading for directed acyclic graph applications based on edge computing in Industrial Internet," *Information Sciences,* vol. 540, pp. 51-68, 2020.

[17] P. Goyal and E. Ferrar, "Graph embedding techniques, applications, and performance: A survey," *Knowledge-Based Systems,* vol. 151, pp. 78-94, 2018.

[18] C.-M. Park, "Universal Multiple Processing Unit Using Graph," in *International Conference on Hybrid Information Technology*, 2012.

[19] V. Agarwal, P. F.P.D and B. D. , "Scalable graph exploration on multi-core processors," in *ACM/IEEE conference on Supercomputing*, 2010.

[20] D. Banerjee, S. Sharma and K. Kothapalli, "Parallel algorithm for large graphs," in *HIPC*, 2013.

[21] R. J. Anderson and G. L. Miller, "A Simple Randomized Parallel Algorithm for List-Ranking," *Information Processing Letters,* vol. 33, no. 5, pp. 269-273, 1990.

[22] D. Barer and C. G. A. fast, "Parallel spanning tree algorithm for symmetric multiprocessor (SMPs)," *Journal of parallel and distributed Computing,* 2005.